

CHAPTER · ENSEMBLE IV

Boosting 모형 튜닝 세 라이브러리 공통 전략

무엇을, 어떤 순서로, 어떤 방법으로 조절할까? 조기 종료 · 하이퍼파라미터 4 그룹 · Random / Bayesian 탐색

“Gradient Boosting의 후예들” 다음 시간

학습 목표

1 튜닝의 원칙

검증 데이터 · CV 로 고르고 , test 는 마지막에 한 번만 쓴다는 원칙을 지킬 수 있다 .

2 하이퍼파라미터 4 그룹

학습률 · 트리 수 , 트리 복잡도 , 무작위성 , 정규화가 bias/variance 에 주는 영향을 설명할 수 있다 .

3 탐색 방법

Grid, Random, Bayesian optimization 의 차이를 이해하고 상황에 맞게 고를 수 있다 .

4 실전 절차

조기 종료를 활용한 단계별 튜닝 절차를 R 로 실행할 수 있다 .

튜닝하면 얼마나 달라질까 ? (직접 실험)

1.51

XGBoost 기본값 (eta 0.3, depth 6, 트리 100 개)

1.34

LightGBM 기본값 (lr 0.1, num_leaves 31, 트리 100 개)

1.16

LightGBM 튜닝 후 (Bayesian 탐색 60 회 + 조기 종료)

테스트 RMSE (작을수록 좋음). 잡음의 표준편차가 1 이므로 이론적 하한 ≈ 1.0

기본값은 “평균적인 데이터” 를 위한 출발점일 뿐 . 이 데이터에서는 튜닝으로 줄일 수 있는 오차 ($RMSE - 1.0$) 가 $0.51 \rightarrow 0.16$ 로 약 68% 감소 . 찾은 설정 : num_leaves = 4 (아주 작은 트리), 강한 L2 정규화 ($\lambda \approx 15$), 부표본 59%

Friedman #1 데이터 (변수 10 개 중 5 개는 잡음), 학습 2,000 · 검증 1,000 · 테스트 5,000. Python 에서 xgboost 3.2 / lightgbm 4.7 / optuna 로 계산

튜닝의 대원칙 : test 는 마지막에 한 번만

Training — 모형 적합

Validation — 튜닝

Test — 최종 평가

또는 Training + Validation 대신 k-fold CV (데이터가 작을 때 권장)

하이퍼파라미터 선택

검증 오차 또는 CV 오차로만 비교한다

조기 종료 (early stopping)

검증 데이터로 판단 . test 로 멈추면 test 가 오염된다

Test set

모든 선택이 끝난 뒤 딱 한 번 . 결과를 보고 다시 튜닝하면 그 test 는 더 이상 “처음 보는 데이터” 가 아니다

최종 모형

고른 설정으로 training(+validation) 전체에 다시 적합한 뒤 test 로 평가

튜닝 결과의 성능을 엄밀하게 보고해야 할 때는 nested CV(바깥 CV 로 평가, 안쪽 CV 로 튜닝) 를 사용

하이퍼파라미터는 네 그룹으로 생각하자

① 학습률 · 트리 수

`learning_rate (eta), n_estimators`

보폭과 걸음 수. 서로 짝을 이룸 → 조기 종료로 트리 수 결정

② 트리 복잡도

`max_depth, num_leaves, min_child_weight / min_data_in_leaf`

트리 하나가 얼마나 복잡한가 → 크면 $\text{bias} \downarrow \text{variance} \uparrow$

③ 무작위성

`subsample (행), colsample_bytree (열)`

매 단계 일부 데이터 · 변수만 사용 → 분산 $\downarrow$, 속도 $\uparrow$

④ 정규화

`lambda (L2), alpha (L1), gamma / min_gain_to_split`

앞 값과 분할에 벌점 → 과대적합 억제

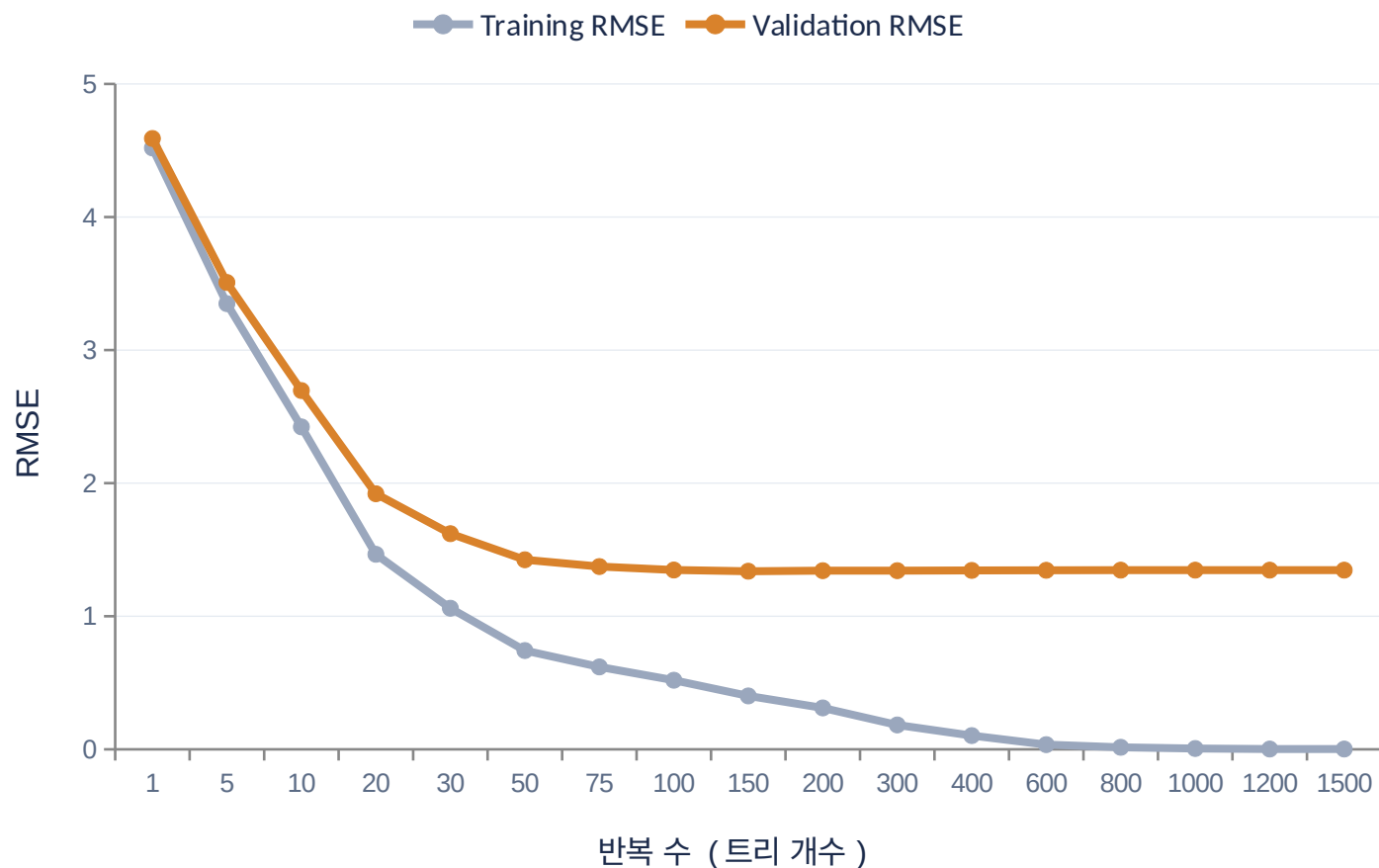
②는 모델을 “키우는” 쪽, ③④는 “억제하는” 쪽. 둘의 균형이 곧 bias-variance 균형

같은 개념 , 다른 이름 : 파라미터 대응표

개념	XGBoost	LightGBM	CatBoost	R gbm
학습률	eta (0.3)	learning_rate (0.1)	learning_rate (자동)	shrinkage
트리 수	nrounds	num_iterations (100)	iterations (1000)	n.trees
트리 크기	max_depth (6)	num_leaves (31), max_depth (-1)	depth (6)	interaction.depth
잎 최소 크기	min_child_weight (1)	min_data_in_leaf (20)	min_data_in_leaf	n.minobsinnode
행 부표본	subsample (1)	bagging_fraction (1) + bagging_freq	subsample / bootstrap_type	bag.fraction
열 부표본	colsample_bytree (1)	feature_fraction (1)	rsm	—
L2 정규화	lambda (1)	lambda_l2 (0)	l2_leaf_reg (3)	—
L1 정규화	alpha (0)	lambda_l1 (0)	—	—
분할 최소 이득	gamma (0)	min_gain_to_split (0)	—	—
조기 종료	early_stopping_rounds	early_stopping_rounds	od_type="Iter", od_wait	gbm.perf()

괄호 안은 기본값 (XGBoost 3.x, LightGBM 4.x, CatBoost 1.2 문서 기준). R 패키지에서는 이름이 조금 다를 수 있음

① 트리 수는 “조기 종료” 로 정한다



XGBoost, eta = 0.1, max_depth = 6. x 축 간격은 균등하지 않음

원리

매 반복마다 검증 오차를 확인하고, k 번 (예 : 50~100 번) 연속으로 좋아지지 않으면 멈춘 뒤 가장 좋았던 반복 수를 사용

이 예제

검증 RMSE 는 136 번째에서 최소 (1.338). 학습 RMSE 는 0 근처까지 계속 감소 → 그 이후는 잡음을 외우는 중

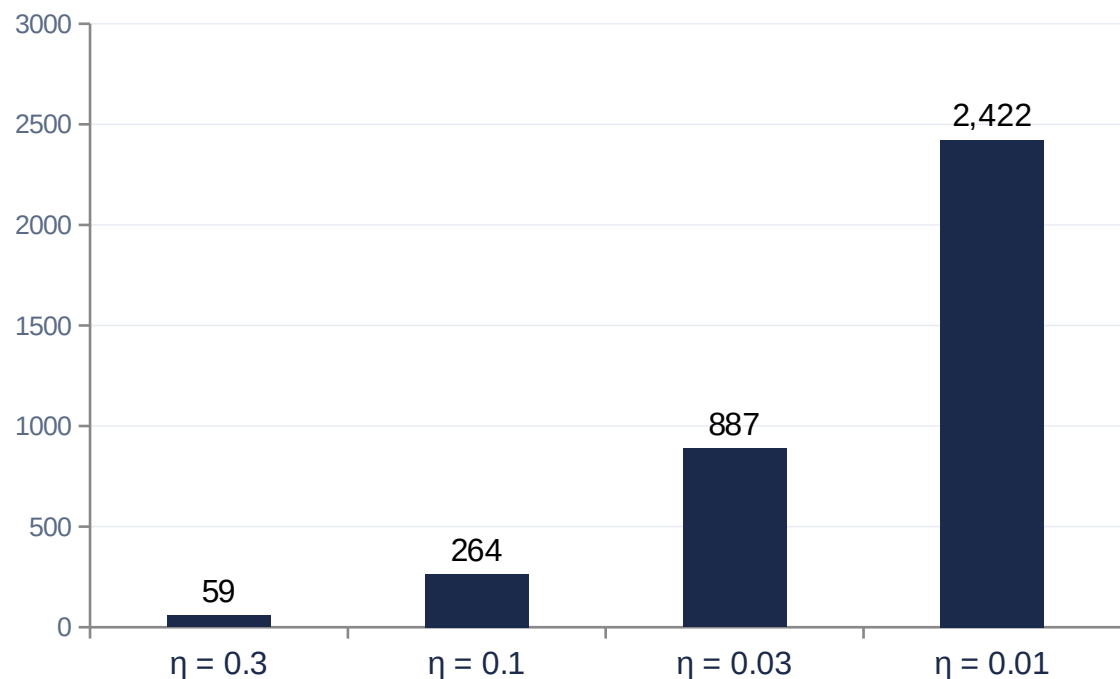
장점

트리 수를 따로 튜닝할 필요가 없고, 시간도 절약 → 트리 수는 넉넉히 (예 : 5,000), 조기 종료가 알아서 결정

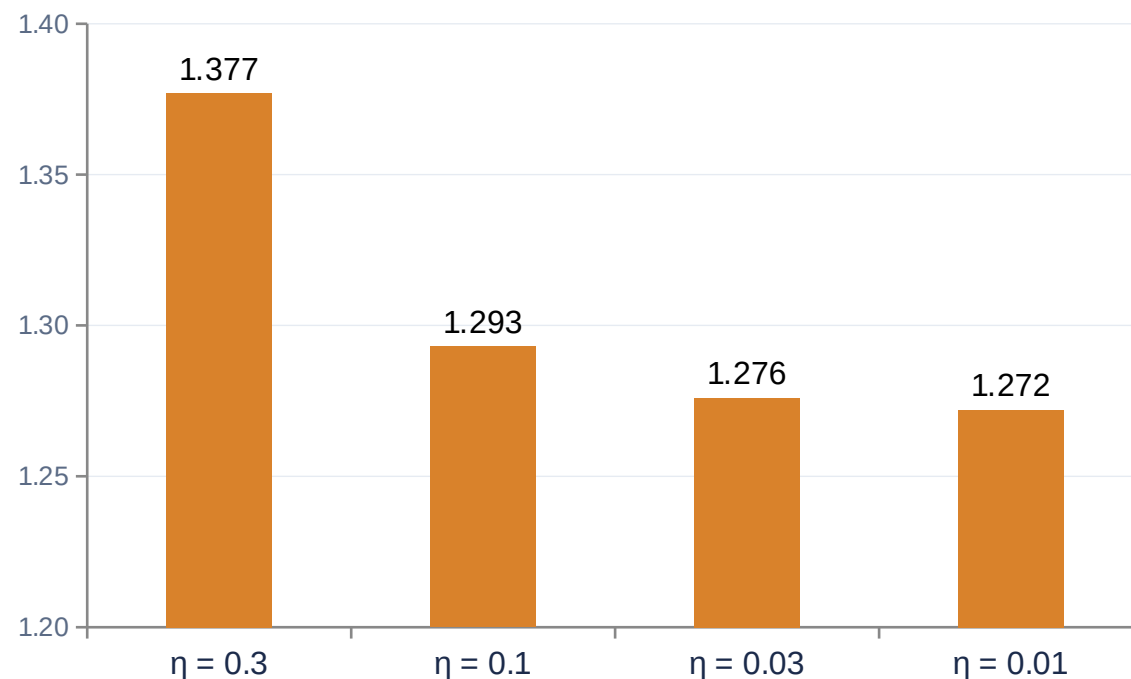
① 학습률과 트리 수 : 작은 보폭, 많은 걸음

XGBoost, max_depth = 4, 조기 종료 100 회

조기 종료가 고른 트리 수

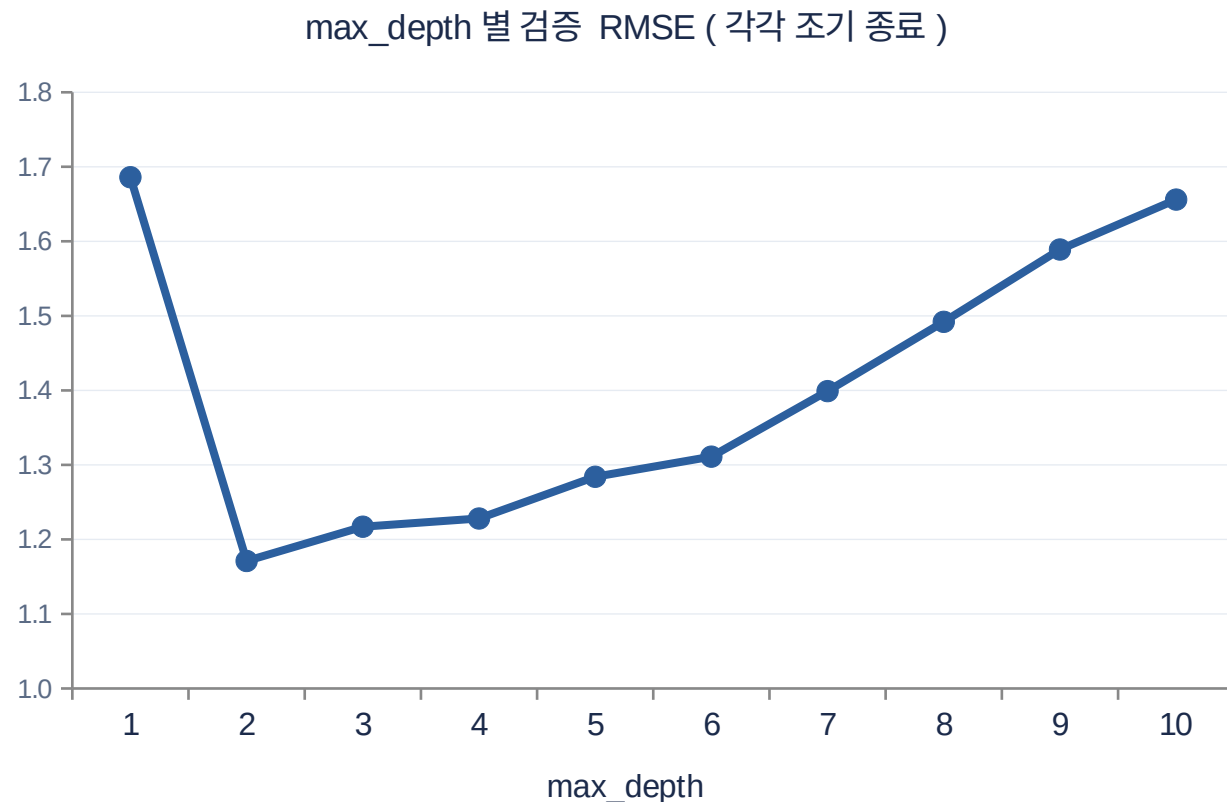


테스트 RMSE (y 축이 1.2 부터 시작)



실전 규칙 : 탐색할 때는 0.05~0.1 로 빠르게 → 설정이 정해지면 0.01~0.03 으로 낮추고 조기 종료로 트리 수를 다시 정한다 .
0.03 → 0.01 은 시간만 3 배 , 이득은 거의 없음

② 트리 복잡도 : 깊을수록 좋은 것이 아니다



이 데이터의 답 : 깊이 2

깊이 1(stump)은 상호작용 ($x_1 \cdot x_2$)을 못 잡아 부족, 3 이상은 점점 과대적합. 참모형의 상호작용 차수가 낮기 때문

깊이 ↔ 잎 수

깊이 d 트리의 잎은 최대 2^d 개. LightGBM은 num_leaves로 직접 조절 → $\text{num_leaves} < 2^{\text{max_depth}}$ 가 되게

잎 최소 크기

min_child_weight(XGB), min_data_in_leaf(LGB): 크게 하면 작은 잎이 안 생겨 과대적합 ↓. 작은 데이터일수록 중요

XGBoost, eta = 0.05. 흔한 탐색 범위: max_depth 3~10, num_leaves 15~255 (로그 척도)

③ 무작위성 : 일부 행 · 일부 열만 사용

행 부표본 (subsample, bagging_fraction)

- 트리마다 학습 데이터의 일부 (예 : 70~90%) 만 비복원추출
- Friedman 의 Stochastic Gradient Boosting (2002)
- 트리들이 조금씩 달라져 분산 ↓ + 계산 ↓
- LightGBM 은 bagging_freq > 0 이어야 적용

열 부표본 (colsample_bytree, feature_fraction)

- 트리 (또는 분할) 마다 변수의 일부만 사용
- Random Forest 의 mtry 와 같은 발상 : 강한 변수의 독점을 막아 트리 간 상관 ↓
- 변수가 많고 서로 상관이 높을 때 효과적
- XGBoost: colsample_bylevel, bynode 도 가능

흔한 탐색 범위 : 두 비율 모두 0.5 ~ 1.0. 너무 작으면 (< 0.5) 개별 트리가 약해져 bias ↑. 보통 0.7~0.9 근처에서 좋은 값이 나온다

Bagging · RF 강의의 “다양성(낮은 상관)” 원리가 boosting 안에서도 그대로 작동

④ 정규화 : 잎 값과 분할에 벌점을

$$w_j^* = -G_j / (H_j + \lambda) \quad \text{분할은 Gain} > \gamma \text{ 일 때만}$$

 λ (L2)

`lambda · lambda_l2 · l2_leaf_reg`

잎 값을 0 쪽으로 줄인다. 관측치가 적은
잎일수록 (H 작음) 강하게 줄어듦. 범위
 $1e-3 \sim 100$ (로그 척도)

 α (L1)

`alpha · lambda_l1`

잎 값을 정확히 0 으로 만들 수 있음. 변수가
매우 많고 희소할 때. 범위 $1e-3 \sim 10$
(로그)

 γ

`gamma · min_gain_to_split`

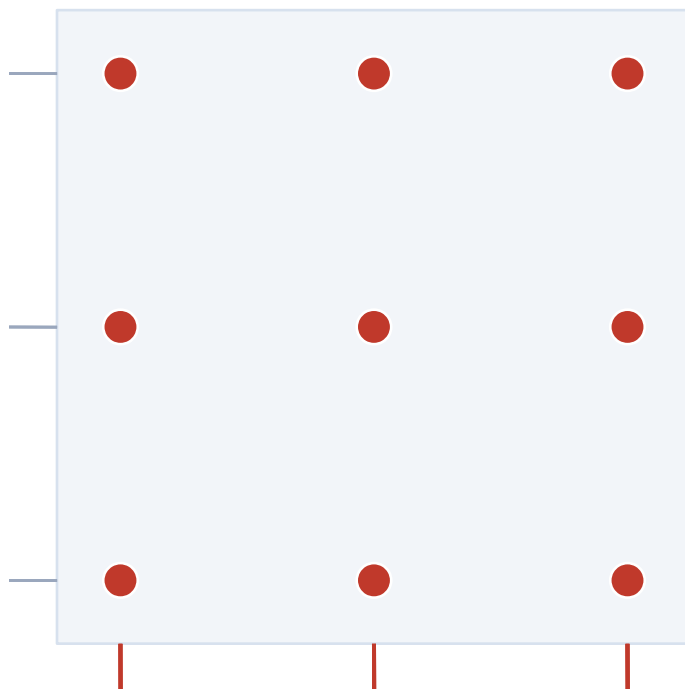
이득이 γ 보다 작은 분할은 하지 않음 →
트리가 작아짐. 범위 $0 \sim 5$ (손실 척도에
따라 다름)

정규화는 보통 마지막에 “미세 조정”. 앞 슬라이드 실험에서도 튜닝 결과 $\lambda \approx 15$ 로 강한 정규화가 선택됨

탐색 방법 : Grid search vs Random search

Grid search: 9 번 시도

↑ 덜 중요한 파라미터

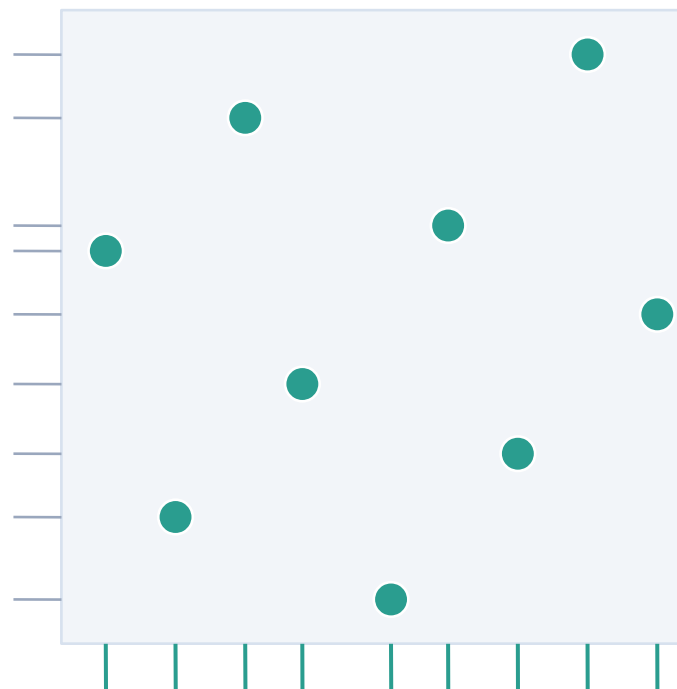


중요한 파라미터 (예 : 학습률)

→ 중요한 축에서는
3 개 값만 시험

Random search: 9 번 시도

↑ 덜 중요한 파라미터



중요한 파라미터 (예 : 학습률)

→ 중요한 축에서도
9 개 값을 시험

파라미터 중 몇 개만 중요한 경우가 대부분 → 같은 예산이면 Random search 가 더 효율적 (Bergstra & Bengio, 2012)

Bayesian optimization: 지난 결과를 보고 다음을 고른다

1 시도

후보 설정으로 모델을 적합하고 CV/ 검증 오차 측정

2 대리 모형 갱신

지금까지의 (설정, 오차) 쌍으로 “설정 → 오차” 를 근사하는 모형 (Gaussian process, TPE 등) 을 적합

3 다음 후보 선택

좋은 것 같은 곳 (exploitation) 과 아직 모르는 곳 (exploration) 의 균형 = acquisition function

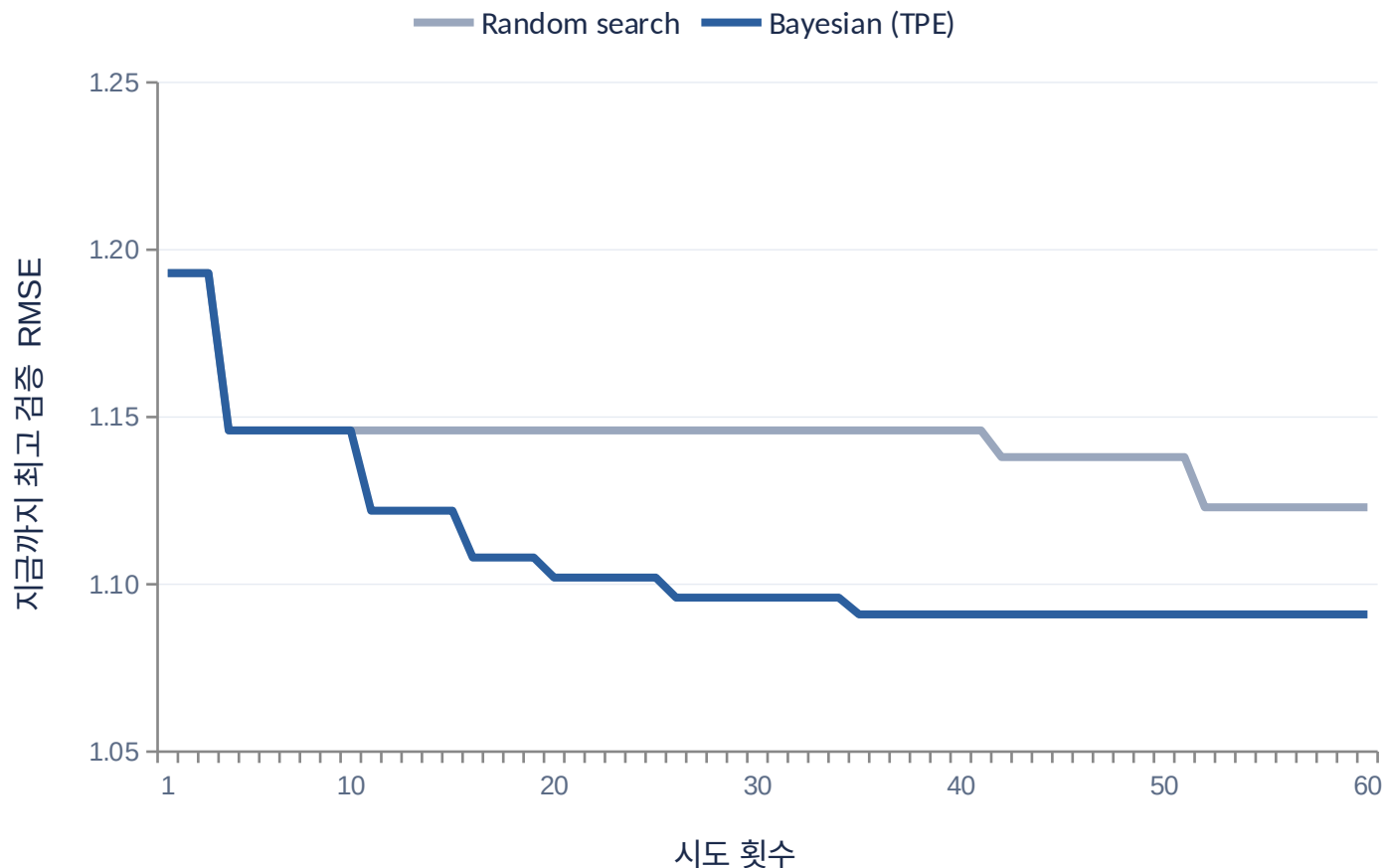
☞ 정해진 횟수만큼 반복

언제 쓰나? 모형 한 번 적합이 오래 걸려서 시도 횟수가 제한될 때 (수십 회 수준)

도구: R — tidymodels tune_bayes(), mlr3mbo, ParBayesianOptimization. Python — Optuna(TPE), scikit-optimize

Grid · Random 은 서로 독립적인 시도 (병렬화 쉬움), Bayesian 은 순차적 — boosting 과 bagging 의 관계와 닮았다

Random vs Bayesian(TPE): 같은 60 번의 시도 (직접 실험)



처음 10 번

둘 다 무작위로 시작 (TPE 도 초기 탐색은 random)

그 이후

TPE 는 좋은 영역에 집중해 빠르게 개선 : 20 번째 1.102, 60 번째 1.091. Random 은 1.123

해석

한 번의 실험 . 시도 횟수가 많아지면 차이는 줄어든다 .
“ 적은 예산에서 Bayesian 이 유리”

LightGBM, 학습률 0.05 고정 + 조기 종료. 탐색: num_leaves, min_data_in_leaf, 행·열 부표본, λ (5 개). Optuna RandomSampler vs TPESampler

실전 튜닝 절차 (추천)

1 출발점

학습률 0.1, 트리 수 넉넉히 + 조기 종료 . 나머지는 기본값 → 기준 성능 기록

2 트리 복잡도

max_depth / num_leaves 와 잎 최소 크기를 먼저 (가장 영향이 큼)

3 무작위성

subsample, colsample 을 0.5~1.0 에서

4 정규화

λ , α , γ 를 로그 척도로 미세 조정

5 학습률 낮추기

0.01~0.05 로 낮추고 조기 종료로 트리 수를 다시 결정

6 최종 적합 · 평가

정한 설정으로 전체 학습 데이터에 적합 → test 로 한 번 평가

2~4 단계는 한꺼번에 Random/Bayesian 탐색으로 해도 된다 . 시간이 부족하면 2 단계만 해도 효과의 대부분을 얻는다

탐색 범위 추천 (출발점)

파라미터	추천 범위	척도	비고
learning_rate	0.01 ~ 0.3	로그	탐색 땀 고정 (0.05~0.1) 하는 경우가 많음
max_depth	3 ~ 10	정수	XGBoost · CatBoost(depth 4~10)
num_leaves	15 ~ 255	로그 · 정수	LightGBM, $< 2^{\text{max_depth}}$
min_child_weight / min_data_in_leaf	1 ~ 100 / 5 ~ 200	로그	데이터가 작을수록 크게
subsample / bagging_fraction	0.5 ~ 1.0	선형	LightGBM 은 bagging_freq=1
colsample_bytree / feature_fraction	0.5 ~ 1.0	선형	변수가 많을수록 작게
lambda / l2_leaf_reg	1e-3 ~ 100	로그	CatBoost 기본 3
alpha / lambda_l1	1e-3 ~ 10	로그	희소 데이터에서
gamma / min_gain_to_split	0 ~ 5	선형	손실 척도에 의존

“로그 척도” : 0.001, 0.01, 0.1, 1 처럼 배수로 고르게 탐색. 범위는 데이터에 따라 조정하는 출발점일 뿐

라이브러리별 튜닝 팁

XGBoost

- max_depth · min_child_weight 먼저
- gamma 는 0 에서 시작해 필요할 때만
- tree_method="hist"(기본) 로 빠르게
- 불균형 분류 : scale_pos_weight

LightGBM

- num_leaves 가 핵심 , max_depth 로 상한
- 작은 데이터 : min_data_in_leaf ↑ , num_leaves ↓
- bagging_fraction 은 bagging_freq 와 함께
- 과대적합 시 max_bin ↓ , feature_fraction ↓

CatBoost

- 기본값이 좋으니 먼저 그대로 시도
- depth(4~10), learning_rate, l2_leaf_reg 순
- 범주형 열은 cat_features 로 지정만
- 느리면 boosting_type="Plain", GPU 사용

공통 : 학습률은 마지막에 낮추고 , 트리 수는 항상 조기 종료로

R 실습 : xgb.cv + 조기 종료 + Random search

R CONSOLE

```
> library(xgboost); set.seed(1234)
> dtr <- xgb.DMatrix(data.matrix(train[,1:11]), label=train$a1)
> grid <- data.frame(max_depth = sample(3:8, 30, replace=TRUE),
+                   min_child_weight = round(exp(runif(30, log(1), log(50)))),
+                   subsample = runif(30, 0.5, 1), colsample_bytree = runif(30, 0.5, 1),
+                   lambda = exp(runif(30, log(1e-3), log(100))))
> res <- lapply(1:nrow(grid), function(k) {
+   p <- c(list(objective="reg:squarederror", eta=0.05), as.list(grid[k,]))
+   cv <- xgb.cv(params=p, data=dtr, nrounds=5000, nfold=5,
+               early_stopping_rounds=100, verbose=0)
+   c(rmse = min(cv$evaluation_log$test_rmse_mean), nrounds = cv$best_iteration)
+ })
> grid <- cbind(grid, do.call(rbind, res))
> best <- grid[which.min(grid$rmse), ] # 가장 좋은 설정
# 최종 : best 설정으로 전체 train에 nrounds = best$nrounds 적합 → test 평가
```

R 실습 : tidymodels 로 Bayesian 탐색 (LightGBM)

R CONSOLE

```
> library(tidymodels); library(bonsai)
> spec <- boost_tree(trees = 1000, learn_rate = 0.05,
+                   tree_depth = tune(), min_n = tune(),
+                   loss_reduction = tune(), sample_size = tune()) |>
+   set_engine("lightgbm") |> set_mode("regression")
> wf <- workflow() |> add_formula(a1 ~ .) |> add_model(spec)
> folds <- vfold_cv(train, v = 5)
> res <- tune_bayes(wf, resamples = folds, initial = 10, iter = 30,
+                  metrics = metric_set(rmse))
> show_best(res, metric = "rmse")
> best <- select_best(res, metric = "rmse")
> final <- finalize_workflow(wf, best) |> fit(train)
> predict(final, test)
```

boost_tree()

엔진 공통 이름 : tree_depth, min_n,
loss_reduction(y), sample_size, mtry

tune_bayes()

initial: 처음 무작위 시도 수 , iter: Bayesian
반복 수 (Gaussian process)

엔진 교체

set_engine("xgboost") 로 바꾸면 같은 코드로
XGBoost

bonsai 패키지가 LightGBM 엔진을 tidymodels 에 연결. 범주형 변수는 factor 로 두면 됨

흔한 실수

× test 로 튜닝하기

test 성능을 보고 설정을 바꾸는 순간 test 는 검증 데이터가 된다 → 성능을 과대평가

× test 로 조기 종료

early stopping 에 test 를 넣는 것도 같은 실수 . 반드시 validation(또는 CV) 사용

× 학습률 고정한 채 트리 수만 늘리기

학습률 0.3 으로 수천 개 → 금방 과대적합 . 트리 수는 조기 종료에 맡기기

× 너무 넓거나 선형인 범위

λ 를 0~100 선형으로 뽑으면 대부분 큰 값만 시도 → 로그 척도로

× CV 오차의 잡음 무시

0.001 차이는 우연일 수 있음 . 비슷하면 더 단순한 (작은 트리 , 큰 정규화) 설정을

× 데이터 누수

타깃 인코딩 · 결측 대체 등 전처리를 CV 밖에서 전체 데이터로 하면 누수 → 전처리도 fold 안에서

SUMMARY

오늘의 핵심 정리

1 원칙

튜닝과 조기 종료는 validation/CV 로 , test 는 마지막에 한 번

2 네 그룹

학습률 · 트리 수 / 트리 복잡도 / 무작위성 / 정규화 — 키우는 쪽과 억제하는 쪽의 균형

3 탐색

Grid 보다 Random, 예산이 적으면 Bayesian. 범위는 로그 척도로

4 절차

lr 0.1 + 조기 종료 → 복잡도 → 부표본 → 정규화 → lr 낮추고 재적합 → test